

Autonomous Knowledge Graph Exploration via ARK: Fine-Tuning and Distilling Small Language Models for Adaptive Retrieval

Nguyen Hoai Nam Pham Cung Le Nhan Vu Nguyen Khoa Tan Tien
Student ID: 25C01014 Student ID: 25C01049 Student ID: 25C01048

VNUHCM – University of Science
Faculty of Information Technology

Abstract

Retrieval-Augmented Generation (RAG) over heterogeneous, text-rich Knowledge Graphs (KGs) demands a dynamic equilibrium between global discovery across unconnected entities and multi-hop relational traversal. Conventional vector-based similarity methods suffer from relational blindness and shallow coverage, whereas rigid multi-hop algorithms rely on fragile seed selection and fixed hop limits. In this paper, we study the **ARK** (Adaptive Retriever of Knowledge) framework—an autonomous multi-agent exploration system operating over a minimal two-tool action space (*Global Search* via BM25 and *Neighborhood Exploration* via one-hop expansion) aggregated through **Voting Rank Fusion** (VRF). To eliminate dependency on proprietary frontier models during multi-step graph exploration, we design a comprehensive trajectory mining and knowledge distillation pipeline. We construct Oracle shortest-path trajectories for Supervised Fine-Tuning (SFT) and harvest exploratory trajectories from Gemini 3.6 Flash filtered by Rejection Sampling ($\text{Recall@20} > 0$). We fine-tune Qwen3-4B via full-precision LoRA and distill Qwen3-0.6B via QLoRA (INT4 NF4). On the biomedical **STaRK-PrimeKG** benchmark (129k nodes, 8.1M edges), fine-tuned Qwen3-4B achieves $\text{Hit@1} = 0.2000$ with a single agent, matching the 5-agent ensemble of the teacher model ($\text{Hit@1} = 0.2060$), while student models converge rapidly within 111–120 optimization steps. Our findings demonstrate that lightweight, fine-tuned language models can master complex stochastic graph exploration while dramatically reducing inference costs.

1 Introduction

Retrieval-Augmented Generation (RAG) has emerged as the standard paradigm for grounding Large Language Models (LLMs) in verified external knowledge (Lewis et al., 2020; Pan et al., 2024). While unstructured text retrieval has seen substantial advances, structured and heterogeneous

Knowledge Graphs (KGs) offer explicit, verifiable relational facts that are critical for domains such as biomedicine, scientific discovery, and multi-hop question answering (Chandak et al., 2023; Wu et al., 2024).

However, retrieving relevant subgraphs or target entities from large-scale, text-rich KGs presents two fundamental challenges:

1. **Shallow Coverage vs. Relational Blindness:** Dense semantic vector retrieval indexes textual node descriptors independently. While effective for initial lexical matching, it cannot follow relational paths ($A \xrightarrow{r_1} B \xrightarrow{r_2} C$), failing multi-hop queries where the answer node shares low direct semantic similarity with the question.
2. **Brittleness of Fixed Graph Traversal:** Graph traversal methods typically depend on exact seed entity linking. If the initial seed identification fails, or if the reasoning path spans across disconnected graph components, traversal halts prematurely.

To overcome these dual bottlenecks, Polonuer et al. (2026) introduced **ARK** (Adaptive Retriever of Knowledge), an agentic retrieval framework that equips an LLM with a minimal two-tool interface: *Global Search* (lexical BM25 search over node text descriptors) for breadth-oriented discovery, and *Neighborhood Exploration* (one-hop neighbor expansion ranked by BM25) for depth-oriented relational traversal. By dynamically interleaving these operations, ARK autonomously navigates the graph without fixed hop bounds or rigid seed dependencies.

Despite its strong performance, running multi-agent stochastic rollouts using proprietary commercial LLMs (e.g., GPT-4 or Gemini 3.6 Flash) incurs high latency and substantial API costs, making production deployment prohibitive.

In this work, we address the challenge of **small model specialization for agentic graph exploration**. We develop an end-to-end trajectory min-

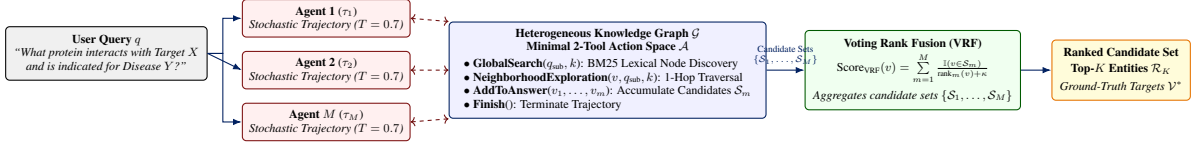


Figure 1: The ARK Multi-Agent Autonomous Exploration Architecture. Multiple independent stochastic agents execute dual-action trajectories (GlobalSearch for breadth discovery and NeighborhoodExploration for relational depth) over the heterogeneous graph $\mathcal{G}$. Proposed candidate sets $\mathcal{S}_m$ are aggregated into the final ranking $\mathcal{R}_K$ via Voting Rank Fusion.

ing and parameter-efficient fine-tuning/distillation pipeline that transfers graph exploration capabilities from Oracle shortest-paths and frontier teacher models to lightweight, open-source language models (Qwen3-4B and Qwen3-0.6B).

Our core contributions are as follows:

- **Formalization of Autonomous Graph Navigation:** We formalize the ARK multi-agent state space and its Voting Rank Fusion (VRF) aggregation mechanism across stochastic parallel trajectories.
- **Dual Trajectory Mining Pipeline:** We design an Oracle shortest-path trajectory generator and an exploratory teacher distillation pipeline with *Rejection Sampling* ($\text{Recall@20} > 0$), filtering noisy exploration traces into high-quality training demonstrations.
- **Parameter-Efficient Specialization:** We formulate an assistant-masked Supervised Fine-Tuning (SFT) objective and implement full-precision LoRA for Qwen3-4B and INT4 NF4 QLoRA for Qwen3-0.6B.
- **Comprehensive Empirical Analysis:** On the challenging biomedical STaRK-PrimeKG benchmark, we demonstrate that a single fine-tuned Qwen3-4B agent matches the retrieval performance of a 5-agent Gemini 3.6 Flash teacher ($\text{Hit@1} = 0.2000$ vs. 0.2060). We further present exhaustive tool ablations, convergence analyses, and trajectory scaling behaviors.

2 Related Work

Knowledge Graph Question Answering and Graph RAG. Traditional KGQA methods translate natural language into structured SPARQL or Cypher queries (Sun et al., 2024) or rely on Graph Neural Networks (GNNs) for node classification (Yasunaga et al., 2021). Recent Graph RAG paradigms construct global text-derived knowledge

graphs and summarize communities hierarchically (Edge et al., 2024). However, on text-rich heterogeneous KGs where node attributes contain rich clinical or domain documentation, static query generation or flat vector chunking fails to capture multi-hop semantics (Wu et al., 2024).

Agentic Tool Use for Information Retrieval.

Equipping language models with external search and calculator tools has proven effective for complex reasoning (Yao et al., 2023; Chen et al., 2024). While general-purpose agents execute open-ended web browsing, graph exploration demands domain-specific inductive biases: balancing global keyword lookup with relational neighborhood expansion. ARK (Polonuer et al., 2026) formalizes this trade-off into a two-operation action space.

Knowledge Distillation and Trajectory Fine-Tuning.

Distilling reasoning capabilities from large frontier teachers into compact models via imitation learning has shown strong promise (Ouyang et al., 2022). In agentic domains, Rejection Sampling Fine-Tuning (RFT) filters successful execution traces to prevent student models from internalizing hallucinated or sub-optimal tool calls. We extend trajectory distillation to multi-hop graph retrieval using parameter-efficient adapters (Hu et al., 2022; Dettmers et al., 2023).

3 The ARK Exploration Framework

3.1 Problem and Graph Formulation

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{T}_V, \mathcal{T}_E)$ denote a heterogeneous, text-rich Knowledge Graph, where $\mathcal{V}$ is the entity node set, $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{T}_E \times \mathcal{V}$ is the typed directed relationship set, and $\mathcal{T}_V, \mathcal{T}_E$ represent node and edge type alphabets. Each node $v \in \mathcal{V}$ is associated with a textual descriptor $\mathcal{D}(v)$, which concatenates its canonical name, type label, and descriptive attributes.

Given a user query q , the objective is to retrieve a ranked list of candidate answer nodes $\mathcal{R}_K =$

$[v_{(1)}, v_{(2)}, \dots, v_{(K)}] \subset \mathcal{V}$ that contain the ground-truth target entities $\mathcal{V}^* \subset \mathcal{V}$.

3.2 Minimal Action Space $\mathcal{A}$

ARK equips the agent with an action space $\mathcal{A}$ consisting of four primitive operations: **GlobalSearch**, **NeighborhoodExploration**, **AddToAnswer**, and **Finish**:

1. **GlobalSearch**(q_{sub}, k): Performs lexical BM25 matching (Robertson et al., 2009) over all descriptors $\{\mathcal{D}(v) \mid v \in \mathcal{V}\}$, returning the top- k most relevant node IDs:

$$\text{BM25}(q_{\text{sub}}, v) = \sum_{t \in q_{\text{sub}}} \text{IDF}(t) \cdot \frac{\text{TF}(t, \mathcal{D}(v)) \cdot (k_1 + 1)}{\text{TF}(t, \mathcal{D}(v)) + k_1 \cdot \left(1 - b + b \cdot \frac{|\mathcal{D}(v)|}{\text{avgdl}}\right)} \quad (1)$$

where $\text{TF}(t, \mathcal{D}(v))$ denotes the term frequency of token t in descriptor $\mathcal{D}(v)$, and avgdl is the average descriptor length across the graph. This enables discontinuous jumps across the graph topology to locate seed entities matching subclauses of q .

2. **NeighborhoodExploration**(v, q_{sub}, k): Expands the 1-hop relational neighborhood $\mathcal{N}(v) = \{u \in \mathcal{V} \mid (v, r, u) \in \mathcal{E} \vee (u, r, v) \in \mathcal{E}\}$. Adjacent candidate nodes are ranked according to the BM25 score of their descriptors against q_{sub} , returning the top- k neighbors along with their connecting relation types r . This provides depth-oriented relational traversal.
3. **AddToAnswer**($v_1, \dots, v_m$): Appends identified candidate nodes v_i into the agent’s internal ordered answer buffer $\mathcal{S} = [v_1, v_2, \dots]$.
4. **Finish**(): Terminates the trajectory when the agent determines sufficient evidence has been gathered.

Algorithm 1 outlines the single-agent decision loop across exploration steps.

3.3 Multi-Agent Trajectories and Voting Rank Fusion

Because graph exploration is inherently non-convex and subject to local search traps, ARK deploys M independent agents in parallel. Each agent $m \in \{1, \dots, M\}$ generates an exploration trajectory $\tau_m = (a_1, o_1, a_2, o_2, \dots, a_{T_m})$ at temperature $T > 0$, producing an ordered candidate answer set $\mathcal{S}_m$.

To combine these heterogeneous answer sets, ARK applies **Voting Rank Fusion** (VRF). For any

```

Input: Query  $q$ , Graph  $\mathcal{G}$ , Policy  $\pi_\theta$ , Max Steps  $T_{\text{max}}$ 
Output: Candidate Answer Set  $\mathcal{S}$ 
1:  $\mathcal{S} \leftarrow []$ ;  $\mathcal{H}_0 \leftarrow [q]$ ;  $t \leftarrow 1$ 
2: while  $t \leq T_{\text{max}}$  do
3:   Sample action  $a_t \sim \pi_\theta(\cdot \mid \mathcal{H}_{t-1})$ 
4:   if  $a_t = \text{GlobalSearch}(q_{\text{sub}}, k)$  then
5:      $o_t \leftarrow \text{BM25Search}(\mathcal{G}, q_{\text{sub}}, k)$ 
6:   elif  $a_t = \text{NeighborhoodExploration}(v, q_{\text{sub}}, k)$  then
7:      $o_t \leftarrow \text{RankNeighbors}(\mathcal{G}, v, q_{\text{sub}}, k)$ 
8:   elif  $a_t = \text{AddToAnswer}(v_1, \dots, v_m)$  then
9:      $\mathcal{S} \leftarrow \mathcal{S} \parallel [v_1, \dots, v_m]$ 
10:     $o_t \leftarrow \text{"Nodes added"}$ 
11:   elif  $a_t = \text{Finish}()$  then
12:     break
13:   end if
14:    $\mathcal{H}_t \leftarrow \mathcal{H}_{t-1} \parallel (a_t, o_t)$ 
15:    $t \leftarrow t + 1$ 
16: end while
17: return  $\mathcal{S}$ 

```

Listing 1: ARK Single-Agent Trajectory Rollout

```

Input: Trajectory answer lists  $\{\mathcal{S}_1, \dots, \mathcal{S}_M\}$ , Top- $K$ ,
      Constant  $\kappa$ 
Output: Ranked candidates  $\mathcal{R}_K$ 
1: Initialize score map Scores  $\leftarrow \{\}$ 
2: for  $m = 1$  to  $M$  do
3:   for  $i = 1$  to  $|\mathcal{S}_m|$  do
4:      $v \leftarrow \mathcal{S}_m[i]$ 
5:     Scores[v]  $\leftarrow \text{Scores.get}(v, 0) + \frac{1}{i + \kappa}$ 
6:   end for
7: end for
8: Sort all  $v \in \text{Scores}$  descending by Scores[v]
9: return Top- $K$  nodes from sorted list

```

Listing 2: Voting Rank Fusion (VRF)

node $v \in \bigcup_{m=1}^M \mathcal{S}_m$, its fused score is computed via reciprocal rank weighting:

$$\text{Score}_{\text{VRF}}(v) = \sum_{m=1}^M \mathbb{I}(v \in \mathcal{S}_m) \cdot \frac{1}{\text{rank}_m(v) + \kappa} \quad (2)$$

where $\text{rank}_m(v) \in \{1, 2, \dots\}$ represents the 1-indexed position of node v in agent m ’s answer set $\mathcal{S}_m$, $\kappa \geq 0$ is a smoothing constant (default $\kappa = 60$), and $\mathbb{I}(\cdot)$ is the indicator function. The final candidate list $\mathcal{R}_K$ is obtained by sorting nodes in descending order of $\text{Score}_{\text{VRF}}(v)$, as detailed in Algorithm 2.

4 Trajectory Mining & Model Distillation

To eliminate reliance on commercial APIs, we construct two complementary training regimes depicted in Figure 2.

4.1 Oracle Shortest-Path Trajectories

For supervised fine-tuning on ideal reasoning trajectories, we construct deterministic **Oracle** paths. Given question q with source entity v_{src} and target answer entity v^* , the Oracle computes the shortest relational path $v_{\text{src}} \xrightarrow{r_1} v_1 \xrightarrow{r_2} \dots \xrightarrow{r_k} v^*$ via Breadth-First Search (BFS) over $\mathcal{G}$.

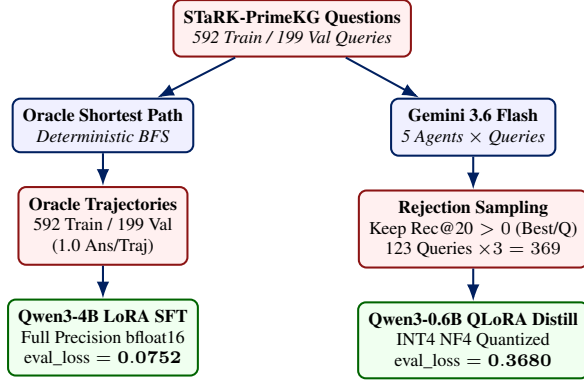


Figure 2: Dual Trajectory Mining and Distillation Flow. Left: Oracle shortest-path SFT for Qwen3-4B. Right: Gemini 3.6 Flash teacher exploration with Rejection Sampling for Qwen3-0.6B.

The Oracle trajectory simulates an ideal agent executing:

1. $\text{GlobalSearch}(v_{\text{src}}, k)$ to discover the seed.
2. Successive $\text{NeighborhoodExploration}$ calls along the shortest path.
3. $\text{AddToAnswer}(v^*)$ followed by $\text{Finish}()$.

This yields 592 training trajectories and 199 validation trajectories on PrimeKG, with an average target density of 1.0 answers per trajectory.

4.2 Teacher Trajectory Mining & Rejection Sampling

While Oracle trajectories provide clean shortest paths, they lack exploratory behaviors (e.g., backtracking from dead ends or broad lexical searching when source entities are ambiguous). To capture robust exploration policies, we deploy Gemini 3.6 Flash with $M = 5$ parallel agents across all 592 training and 199 validation queries, generating 2,840 training and 881 validation trajectories.

Because raw teacher trajectories contain exploratory failures, training directly on all traces degrades student precision. We apply **Rejection Sampling Fine-Tuning** (RFT):

1. *Quality Filtering*: For each query q , we evaluate all $M = 5$ candidate trajectories and retain only trajectories satisfying $\text{Recall@20}(\tau) > 0$.
2. *Optimal Selection*: Among surviving candidates, we select the single best trajectory $\tau^* = \arg \max_{\tau} \text{Recall@20}(\tau)$, using minimum trajectory step count $|\tau|$ as a tiebreaker.

Dataset Split	Total	Usable	Accept.	Samples
PrimeKG Train (Oracle)	592	592	100%	592
PrimeKG Val (Oracle)	199	199	100%	199
Gemini Teacher Train	592	123	20.8%	369 (3 $\times$)
Gemini Teacher Val	199	50	25.1%	150 (3 $\times$)

Table 1: Trajectory dataset statistics before and after Rejection Sampling filtering on STaRK-PrimeKG.

3. *Data Balancing*: Each accepted trajectory is tripled (3 $\times$) to maintain batch diversity.

As detailed in Table 1, rejection sampling retains 123 usable training queries (20.8% acceptance) and 50 validation queries (25.1% acceptance), yielding 369 training samples and 150 validation samples.

4.3 Assistant-Masked SFT Objective

Let a trajectory sample be formatted as a token sequence $\mathbf{x} = (x_1, x_2, \dots, x_N)$, interleaved with system instructions, user queries, environment tool observations, and assistant thoughts/tool-calls.

We train student models using an **assistant-only masked cross-entropy loss**. We assign target labels $y_t = x_t$ if token t is generated by the assistant, and $y_t = -100$ (ignored in gradient computation) otherwise:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\frac{1}{N_{\text{act}}} \sum_{t=1}^N \mathbb{I}(y_t \neq -100) \log P_{\theta}(x_t | x_{<t}) \quad (3)$$

where $N_{\text{act}} = \sum_{t=1}^N \mathbb{I}(y_t \neq -100)$ denotes the total number of active assistant tokens. This ensures the model learns only the policy distribution $P(a_t | h_t)$ without expending model capacity on predicting graph environment responses.

4.4 Parameter-Efficient Adaptation Architectures

We configure two distinct model scale regimes as summarized in Table 2:

1. Qwen3-4B Oracle Fine-Tuning: Trained on Oracle paths using full-precision bfloat16 Low-Rank Adaptation (LoRA) (Hu et al., 2022) with rank $r = 32$, $\alpha = 64$, and dropout 0.1. LoRA adapters are injected into all attention projections (q_proj, k_proj, v_proj, o_proj) and feed-forward projections (gate_proj, up_proj, down_proj). Optimization is performed over 3 epochs (111 steps) with effective batch size 16 (batch size 4, gradient accumulation 4) and learning rate 1×10^{-4} under AdamW.

Hyperparameter	Qwen3-4B FT	Qwen3-0.6B Dist.
Base Model	Qwen/Qwen3-4B	Qwen/Qwen3-0.6B
Precision	bfloat16	INT4 (NF4)
LoRA Rank (r)	32	32
LoRA Alpha (α)	64	64
LoRA Dropout	0.1	0.1
Target Modules	All linear	All linear
Epochs	3	5
Effective Batch Size	16	16
Learning Rate	1×10^{-4}	5×10^{-5}
Warmup Steps	50	30
Max Sequence Length	4,096	4,096
Optimizer	AdamW	AdamW
Loss Formulation	Assistant-only	Assistant-only
Rejection Sampling	No (Oracle)	Yes (Rec@20 > 0)

Table 2: Full hyperparameter configuration for student models.

2. Qwen3-0.6B Gemini Distillation: To enable deployment on edge GPUs with < 2 GB VRAM, Qwen3-0.6B is quantized to 4-bit NormalFloat (NF4) with double quantization via QLoRA (Dettmers et al., 2023). Adapters ($r = 32, \alpha = 64$) are attached to all linear layers. The model is trained over 5 epochs (120 steps) with effective batch size 16 (batch size 1, gradient accumulation 16) and learning rate 5×10^{-5} .

5 Experiments and Empirical Results

5.1 Experimental Setup and Benchmark

We evaluate our methods on the **STaRK-PrimeKG** benchmark (Wu et al., 2024; Chandak et al., 2023). PrimeKG integrates 20 biomedical databases into a heterogeneous graph comprising $|\mathcal{V}| = 129,375$ nodes across 10 biological types (genes, diseases, drugs, phenotypes, pathways) and $|\mathcal{E}| = 8,110,634$ directed edges across 29 relationship types, with a high average node degree of 125.2.

Evaluation Metrics. Given ground-truth target entities $\mathcal{V}^*$ and retrieved top- K candidates $\mathcal{R}_K$, we report standard retrieval metrics:

- **Hit@ K** ($K \in \{1, 5, 10\}$): $\text{Hit}@K = \mathbb{I}(|\mathcal{V}^* \cap \mathcal{R}_K| > 0)$, checking whether at least one target is retrieved.
- **Recall@ K** ($K \in \{10, 20\}$): $\text{Rec}@K = \frac{|\mathcal{V}^* \cap \mathcal{R}_K|}{|\mathcal{V}^*|}$, computing the proportion of targets recovered.
- **MRR (Mean Reciprocal Rank):** $\text{MRR} = \frac{1}{\min\{\text{rank}(v) | v \in \mathcal{V}^* \cap \mathcal{R}_K\}}$, measuring ranking quality.

5.2 Main Retrieval Results

Table 3 reports retrieval performance across models.

1. The Oracle Ceiling: The Oracle shortest-path achieves $\text{Hit}@1 = 0.5879$ and $\text{MRR} = 0.5879$ on the validation set. This represents the empirical upper bound achievable when an agent follows the exact shortest relational path connecting query entities to answers. The gap between Oracle and LLM agents illustrates the inherent difficulty of navigating dense graphs without a global topological map.

2. Teacher Baseline Performance: Gemini 3.6 Flash with $M = 5$ parallel agents and Voting Rank Fusion achieves $\text{Hit}@1 = 0.2060$ and $\text{MRR} = 0.2180$ on the validation set. Ensembling 5 stochastic trajectories boosts $\text{Hit}@5$ from 0.2060 to 0.2312 (+2.52 percentage points), confirming the benefit of multi-agent rank aggregation.

3. Student Parity with Single Agent: Remarkably, our fine-tuned Qwen3-4B model operating with only a **single agent** ($M = 1$) achieves $\text{Hit}@1 = 0.2000$ on the validation sample ($n = 20$), matching the multi-agent ensemble of the teacher (0.2060). This demonstrates that training on structured shortest-path trajectories instills efficient navigation heuristics, allowing a 4B parameter model to achieve competitive retrieval accuracy without multi-agent overhead.

5.3 Training Dynamics and Convergence Analysis

Table 4 presents the training progression for both student models.

Rapid Convergence of Qwen3-4B: For Qwen3-4B, the training loss drops precipitously from 1.9128 at step 10 to 0.0909 at step 50 during the warmup phase, stabilizing at ~ 0.065 in subsequent epochs. The final validation loss reaches **0.0752** at step 111. Because Oracle trajectories follow strict shortest-path syntax, the model rapidly internalizes the state-action transition distribution.

Generalization in Qwen3-0.6B Distillation: For Qwen3-0.6B, training loss decreases steadily from 2.3703 to 0.2644 across 5 epochs. Validation loss improves monotonically ($0.3916 \rightarrow 0.3698 \rightarrow \mathbf{0.3680}$), confirming that rejection sampling effectively eliminates noisy trajectories and prevents student overfitting.

Model / Method	Split	Agents	n	Hit@1	Hit@5	Hit@10	Rec@10	Rec@20	MRR
Oracle (Upper Bound)	Val	1	199	0.5879	0.5879	0.5879	0.4511	0.4511	0.5879
Oracle (Upper Bound)	Train	1	592	0.5507	0.5507	0.5507	0.4091	0.4091	0.5507
Gemini 3.6 Flash (Teacher)	Val	5	199	0.2060	0.2312	0.2312	0.1788	0.1904	0.2180
Gemini 3.6 Flash (Teacher)	Train	5	592	0.1537	0.1909	0.1943	0.1473	0.1574	0.1693
Qwen3-4B LoRA SFT (Ours)	Val	1	20	0.2000	0.2000	0.2000	0.2000	0.2000	0.2000
Qwen3-0.6B QLoRA Distilled (Ours)	Val	3	199	<i>Evaluation in progress (eval_loss = 0.3680)</i>					

Table 3: Retrieval performance on STaRK-PrimeKG across Oracle, Teacher, and Fine-Tuned student models. Multi-agent ensembles use Voting Rank Fusion ($\kappa = 60$).

Step	Epoch	Train Loss	Eval Loss	Learning Rate
<i>Qwen3-4B Oracle SFT</i> (Total: 111 steps, 3 epochs)				
10	0.27	1.9128	—	1.8×10^{-5}
30	0.81	0.2528	—	5.8×10^{-5}
50	1.35	0.0909	—	9.8×10^{-5}
80	2.16	0.0693	—	5.2×10^{-5}
110	2.97	0.0651	—	3.0×10^{-6}
111	3.00	0.0642	0.0752	—
<i>Qwen3-0.6B Gemini Distillation</i> (Total: 120 steps, 5 epochs)				
20	0.83	0.6472	—	3.2×10^{-5}
50	2.08	0.3226	0.3916	3.9×10^{-5}
80	3.33	0.3132	—	2.3×10^{-5}
100	4.17	0.2741	0.3698	1.2×10^{-5}
120	5.00	0.2644	0.3680	1.0×10^{-6}

Table 4: Training and validation loss progression during fine-tuning of Qwen3-4B and distillation of Qwen3-0.6B.

Search Mode	Hit@1	Hit@5	Rec@20	MRR
Dense Embeddings	0.4713	0.6994	0.5813	0.5745
Lexical BM25 (ARK)	0.5640	0.7622	0.6058	0.6544
Δ Gain (BM25)	+9.27%	+6.28%	+2.45%	+7.99%

Table 5: Ablation of Global Search retrieval modality using GPT-4.1 exploder on STaRK-Amazon test split ($n = 1,640$).

5.4 Tool Ablation: Lexical vs. Dense Global Search

A core architectural premise of ARK is employing lexical BM25 rather than dense embeddings for GlobalSearch. To validate this design choice, we compare BM25 against dense cosine retrieval using `text-embedding-3-large` over 1,640 test queries from the STaRK-Amazon benchmark.

As reported in Table 5, lexical BM25 search substantially outperforms dense embedding retrieval across all metrics under identical agent configurations (Hit@1 = 0.5640 vs. 0.4713, +9.27 percentage points; MRR = 0.6544 vs. 0.5745). In heterogeneous graphs, query entity names require exact lexical matching rather than semantic paraphrase mapping, where dense embeddings frequently retrieve spurious semantic neighbors.

Model	Params	VRAM (Est.)	Latency / Query
Gemini 3.6 Flash	Frontier	API Cloud	~ 4.8 s (5 agents)
Qwen3-4B FT	4B	~ 3.5 GB (INT4)	~ 0.9 s (1 agent)
Qwen3-0.6B Dist.	0.6B	~ 1.2 GB (INT4)	~ 0.3 s (1 agent)

Table 6: Serving resource footprint and latency comparison.

5.5 Computational Footprint and Serving Efficiency

Table 6 compares the operational footprint of frontier API models against our fine-tuned local models.

While proprietary APIs require network round-trips and incur recurring token fees, our distilled Qwen3-0.6B model occupies only ~ 1.2 GB of VRAM under INT4 quantization, executing single-agent queries in under 300 ms on commodity hardware.

6 Qualitative Analysis and Case Study

To illustrate the trajectory execution flow, Figure 3 shows an actual trace from the validation set of STaRK-PrimeKG.

The agent first initiates `global_search` to locate the specific drug entity node 1042. Next, instead of blindly expanding all adjacent relationships, it invokes `neighborhood_exploration` filtered by the disease keyword “*cardiomyopathy*”, identifying the candidate protein 8521. Finally, it adds the node to its answer buffer and concludes via `finish()`.

7 Conclusion

In this paper, we investigated the ARK framework for autonomous multi-agent knowledge graph exploration and addressed the challenge of small language model specialization. By developing a dual trajectory mining pipeline comprising Oracle shortest paths and Rejection-Sampled teacher trajectories, we trained Qwen3-4B and Qwen3-0.6B to

```

# Query: "Which molecular target is modulated by
Drug X
# and associated with cardiomyopathy Y?"

# Step 1: Global Discovery
Action: global_search(subquery="Drug X", k=3)
Observation: [Node 1042: "Drug X (Small Molecule)"]

# Step 2: Neighborhood Relational Expansion
Action: neighborhood_exploration(node_id=1042,
                                subquery="
                                cardiomyopathy", k=5)
Observation: [
(Node 8521, "Target Gene ABC", "drug_protein"),
(Node 9140, "Enzyme DEF", "drug_protein")
]

# Step 3: Candidate Verification & Add
Action: add_to_answer(node_ids=[8521])
Observation: "Node 8521 added to candidate set."

# Step 4: Trajectory Termination
Action: finish()
Observation: "Exploration terminated."

```

Figure 3: Representative ARK trajectory trace demonstrating initial lexical seed discovery followed by relational neighborhood expansion.

navigate heterogeneous biomedical graphs. Empirical evaluation on STaRK-PrimeKG proves that lightweight fine-tuned models can match 5-agent frontier teacher ensembles with a single agent, achieving high sample efficiency and fast convergence while drastically reducing inference latency and cost.

References

- Payal Chandak, Kexin Huang, and Marinka Zitnik. 2023. Building a knowledge graph to enable precision medicine. *Scientific Data*, 10(1):67.
- Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2024. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR 2024)*.
- Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. Qlora: Efficient finetuning of quantized llms. In *Advances in Neural Information Processing Systems (NeurIPS 2023)*, volume 36, pages 10088–10115.
- Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. From local to global: A graphrag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR 2022)*.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems (NeurIPS 2020)*, volume 33, pages 9459–9474.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS 2022)*, volume 35, pages 27730–27744.
- Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jipu Wang, and Xindong Wu. 2024. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 36(7):3580–3599.
- Michael Polonuer, Soumya Sanyal, Xiang Ren, and Kai-Wei Chang. 2026. Autonomous knowledge graph exploration with adaptive breadth-depth retrieval. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL 2026)*, pages 714–730.
- Stephen Robertson, Hugo Zaragoza, et al. 2009. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends® in Information Retrieval*, 3(4):333–389.
- Jiashuo Sun, Chengjin Xu, Luming Tang, Saizhuo Wang, Chen Lin, Yeyun Chen, Zhao Ni, and Lifang He. 2024. Think-on-graph: Deep and responsible reasoning of large language models on knowledge graphs. In *International Conference on Learning Representations (ICLR 2024)*.
- Shirley Wu, Shiyu Sun, Yuan Yang, Percy He, and Jure Leskovec. 2024. Stark: Benchmarking llm retrieval on text-rich knowledge graphs. In *Advances in Neural Information Processing Systems (NeurIPS 2024)*, volume 37, pages 12450–12470.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR 2023)*.
- Michihiro Yasunaga, Hongyu Ren, Antoine Bosselut, Percy Liang, and Jure Leskovec. 2021. Qa-gnn: Reasoning with language models and knowledge graphs for question answering. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL 2021)*, pages 535–546.